μP Feature Learning 分析

本 notebook 可复现以下：复现论文 Deep ReLU 网络的 μP 缩放。

实验环境

- 设置 seed=42 以保证可复现性
- 使用 Kaiming Normal 初始化（fan_in 模式）
- 数据：合成 Gaussian 数据，维度 d=512，B=256
- 宽度扫描：width = [128, 256, 512, 1024, 2048]
- 深度：L = 2（两层 ReLU 网络）
- 学习率：η = 0.01（固定）
- 训练：1 step SGD（full-batch）

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from collections import defaultdict

from jax import PyTree, jax_dataclasses, jax_loss_losses, make_loss_objective, MeanAdam

DEVICE=torch.device("cuda:1" if torch.cuda.is_available() else "cpu")
print(f"Device: {DEVICE}")
if DEVICE.type == "cuda":
    print(f"GPU: {torch.cuda.get_device_name(1)}")
torch.manual_seed(42)
np.random.seed(42)
```

```
Device: cuda:1
GPU: NVIDIA A100-SXM4-40GB
```

数据 & 模型（只复方 demo 一致）

```
batch_size = 256
d_in = 512
log_d_range = [6, 9]
n_steps = 1000
lr = 0.01
data_dir = '/home/jiang/muP/data'

torch.manual_seed(42)
X = torch.randn(512, 1, 1, 1).cuda(1) * 3.0 # shape: (B, 1, 1, 1)

Y = torch.sin(X) + 0.1 * torch.randn_like(X)

print(f"Dataset: X.shape={X.shape}, Y.shape={Y.shape}")
print(f"X range: [{X.min:.2f}, {X.max:.2f}]")
print(f"Y range: [{Y.min:.2f}, {Y.max:.2f}]")
print(f"Y mean: {Y.mean:.2f}, Y std: {Y.std:.2f}")

from jax import PyTree, jax_dataclasses, jax_loss_losses, make_loss_objective, MeanAdam

DATA_DIR=f"/home/jiang/muP/data"
train_loader = torch.utils.data.DataLoader(TensorDataset(X, Y), batch_size=512, shuffle=True, num_workers=4, pin_memory=True, persistent_workers=True, prefetch_factor=2)
train_loader = torch.utils.data.DataLoader(TensorDataset(X, Y), batch_size=512, shuffle=True, num_workers=4, pin_memory=True, persistent_workers=True, prefetch_factor=2)

print("Dataset: [-1, 1]^d, Gaussian noise σ=0.1")
print("Loss: MSE")
print("Optimizer: SGD (full-batch)")
```

```
Dataset: X.shape=torch.Size([512, 1, 1, 1]), Y.shape=torch.Size([512, 1, 1, 1])
X range: [-9.47, 8.97]
Y range: [-1.00, 1.00]
Y mean: -0.02, Y std: 0.71
Dataset: [-1, 1]^d, Gaussian noise σ=0.1
Loss: MSE
Optimizer: SGD (full-batch)
```

数据 & 模型（只复方 demo 一致）

```
class MLP(nn.Module):
    def __init__(self, width=512, depth=2, operator=1, mult=1):
        super().__init__()
        self.width = width
        self.depth = depth
        self.operator = operator
        self.mult = mult
        for batch_idx, (data, target) in enumerate(train_loader):
            data = data.cuda(1).reshape(-1, 1)
            target = target.cuda(1)
            optimizer.zero_grad()
            output = model(data)
            loss = criterion(output, target)
            loss.backward()
            optimizer.step()
            if optimizer.step:
                pass
            print(f"Step {batch_idx}: Complete")
            if batch_idx >= 0:
                print("Training complete (1 step)")
                break
        return self.fc1, self.fc2, self.fc3

print("Model: 3-layer MLP")
def forward(self, x):
    x = x.reshape(-1, 1)
    x = self.fc1(x)
    x = F.relu(x)
    x = self.fc2(x)
    x = F.relu(x)
    x = self.fc3(x)
    return x.reshape(-1, 1, 1, 1)
class MLP(nn.Module):
    def __init__(self, width=512, depth=2, operator=1, mult=1, input_dim=1, output_dim=1):
        super().__init__()
        self.width = width
        self.depth = depth
        self.operator = operator
        self.mult = mult
        self.input_dim = input_dim
        self.output_dim = output_dim
        self.fc1 = nn.Linear(input_dim, width, bias=False)
        self.fc2 = nn.Linear(width, width, bias=False)
        self.fc3 = nn.Linear(width, output_dim, bias=False)
        self.initialized = False
        self.init_parameters(operator, mult)
    def init_parameters(self):
        if self.initialized:
            return
        with torch.no_grad():
            nn.init.kaiming_normal_(self.fc1.weight, nonlinearity='relu')
            nn.init.kaiming_normal_(self.fc2.weight, nonlinearity='relu')
            nn.init.kaiming_normal_(self.fc3.weight, nonlinearity='relu')
        self.initialized = True
    def forward(self, x):
        if not self.initialized:
            raise RuntimeError("Call init_parameters() first")
        x = x.reshape(-1, self.input_dim)
        x = self.fc1(x)
        x = F.relu(x)
        x = self.fc2(x)
        x = F.relu(x)
        x = self.fc3(x)
        return x.reshape(-1, self.output_dim)
class MLP(nn.Module):
    def __init__(self, width=512, depth=2, operator=1, mult=1, input_dim=1, output_dim=1):
        super().__init__()
        self.width = width
        self.depth = depth
        self.operator = operator
        self.mult = mult
        self.input_dim = input_dim
        self.output_dim = output_dim
        self.fc1 = nn.Linear(input_dim, width, bias=False)
        self.fc2 = nn.Linear(width, width, bias=False)
        self.fc3 = nn.Linear(width, output_dim, bias=False)
        self.initialized = False
        self.init_parameters(operator, mult)
    def init_parameters(self):
        if self.initialized:
            return
        with torch.no_grad():
            nn.init.kaiming_normal_(self.fc1.weight, nonlinearity='relu')
            nn.init.kaiming_normal_(self.fc2.weight, nonlinearity='relu')
            nn.init.kaiming_normal_(self.fc3.weight, nonlinearity='relu')
        self.initialized = True
    def forward(self, x):
        if not self.initialized:
            raise RuntimeError("Call init_parameters() first")
        x = x.reshape(-1, self.input_dim)
        x = self.fc1(x)
        x = F.relu(x)
        x = self.fc2(x)
        x = F.relu(x)
        x = self.fc3(x)
        return x.reshape(-1, self.output_dim)
```

实验 A: μ(n)初始化 vs Width

- 固定 depth=2
- 扫描 width = [128, 256, 512, 1024, 2048]
- 记录 Input Gram 和 Hidden Gram 的 Frobenius 变化

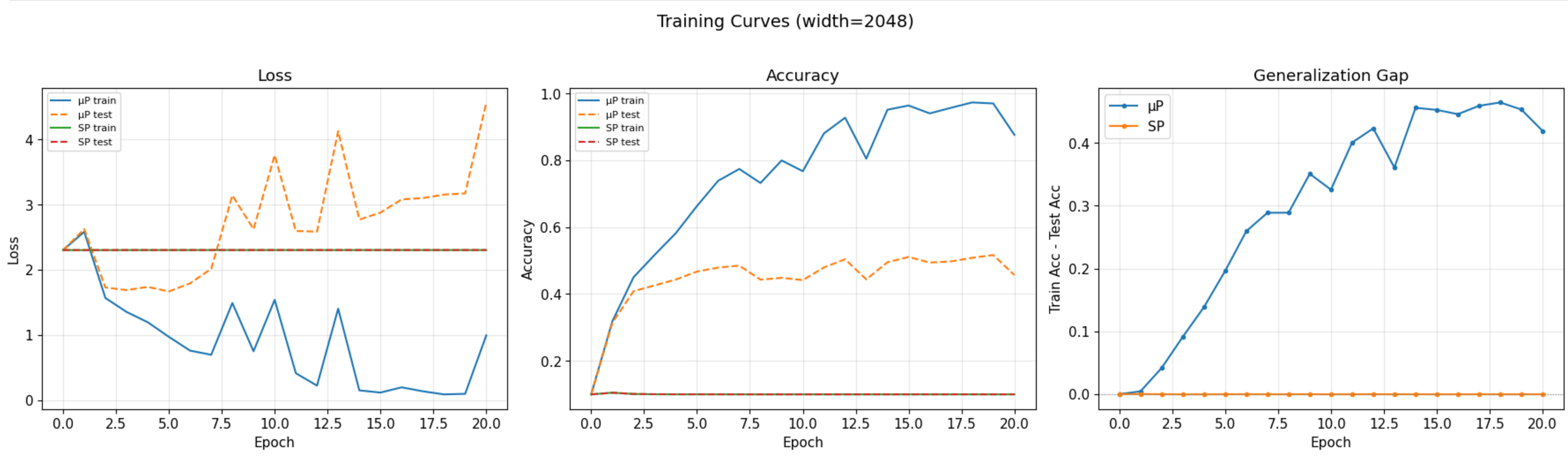
```
def measure_gram(model,batch_size=256, lr=0.1, seed=0):
    torch.manual_seed(seed)
    device = next(model.parameters).device
    X = torch.randn(batch_size, d_in).to(device)
    print("Forward pass...")
    Y = model(X)
    print("Done")

    for p in model:
        if p.grad is None:
            continue
        if 'weight' in name:
            print(f"  {name}: skipping (non-weight)")
            continue
        if p.grad.ndim < 2:
            print(f"  {name}: skipping (ndim={p.grad.ndim})")
            continue
        print(f"  {name}: applying muon_update")
        muon_update = zeropower_via_newtonschulz5(p.grad, steps=5)
        print(f"  {name}: muon_update shape={muon_update.shape}")
        p.data -= lr * muon_update

    W = list(model.parameters)[0]
    print(f"W shape: {W.shape}")

    G = W @ W.T
    print(f"G shape: {G.shape}")

    return G.cpu().detach().numpy()

widths = [128, 256, 512, 1024, 2048]
G_init = {}
for w in widths:
    print(f"\n=== Measuring muon_update(grad) for width={w} ===")
    model = MLP(width=w, depth=2, operator=1, mult=1).cuda(1)
    G = measure_gram(model, lr=0.1, seed=42)
    G_init[w] = G
    print(f"Stored G_init[{w}], shape={G.shape}")
    del model
    torch.cuda.empty_cache()

print("\n=== Summary ===")
for w in widths:
    print(f"  width={w}: G_init[{w}].shape={G_init[w].shape}")
```

```
=== Measuring muon_update(grad) for width=128 ===
Forward pass...
Done
  fc1.weight: applying muon_update
  fc1.weight: muon_update shape=torch.Size([128, 512])
  fc2.weight: applying muon_update
  fc2.weight: muon_update shape=torch.Size([128, 128])
  fc3.weight: applying muon_update
  fc3.weight: muon_update shape=torch.Size([1, 128])
W shape: torch.Size([128, 512])
G shape: torch.Size([128, 128])
Stored G_init[128], shape=(128, 128)

=== Measuring muon_update(grad) for width=256 ===
Forward pass...
Done
  fc1.weight: applying muon_update
  fc1.weight: muon_update shape=torch.Size([256, 512])
  fc2.weight: applying muon_update
  fc2.weight: muon_update shape=torch.Size([256, 256])
  fc3.weight: applying muon_update
  fc3.weight: muon_update shape=torch.Size([1, 256])
W shape: torch.Size([256, 512])
G shape: torch.Size([256, 256])
Stored G_init[256], shape=(256, 256)

=== Measuring muon_update(grad) for width=512 ===
Forward pass...
Done
  fc1.weight: applying muon_update
  fc1.weight: muon_update shape=torch.Size([512, 512])
  fc2.weight: applying muon_update
  fc2.weight: muon_update shape=torch.Size([512, 512])
  fc3.weight: applying muon_update
  fc3.weight: muon_update shape=torch.Size([1, 512])
W shape: torch.Size([512, 512])
G shape: torch.Size([512, 512])
Stored G_init[512], shape=(512, 512)

=== Measuring muon_update(grad) for width=1024 ===
Forward pass...
Done
  fc1.weight: applying muon_update
  fc1.weight: muon_update shape=torch.Size([1024, 512])
  fc2.weight: applying muon_update
  fc2.weight: muon_update shape=torch.Size([1024, 1024])
  fc3.weight: applying muon_update
  fc3.weight: muon_update shape=torch.Size([1, 1024])
W shape: torch.Size([1024, 512])
G shape: torch.Size([1024, 1024])
Stored G_init[1024], shape=(1024, 1024)

=== Measuring muon_update(grad) for width=2048 ===
Forward pass...
Done
  fc1.weight: applying muon_update
  fc1.weight: muon_update shape=torch.Size([2048, 512])
  fc2.weight: applying muon_update
  fc2.weight: muon_update shape=torch.Size([2048, 2048])
  fc3.weight: applying muon_update
  fc3.weight: muon_update shape=torch.Size([1, 2048])
W shape: torch.Size([2048, 512])
G shape: torch.Size([2048, 2048])
Stored G_init[2048], shape=(2048, 2048)

=== Summary ===
  width=128: G_init[128].shape=(128, 128)
  width=256: G_init[256].shape=(256, 256)
  width=512: G_init[512].shape=(512, 512)
  width=1024: G_init[1024].shape=(1024, 1024)
  width=2048: G_init[2048].shape=(2048, 2048)
```

```
 15  0.1149  2.8778  0.963  0.511 +0.453   0.871277
 16  0.1955  3.0801  0.940  0.494 +0.446   0.893446
 17  0.1345  3.1004  0.957  0.497 +0.459   0.913451
 18  0.0871  3.1538  0.973  0.508 +0.465   0.930168
 19  0.0955  3.1727  0.970  0.516 +0.453   0.948158
 20  0.9933  4.5685  0.876  0.457 +0.419   0.965230

=== SP w=2048 lr=0.1 ===
 ep    tr_l    te_l   tr_a   te_a    gap    dW2/W20
  0  2.3026  2.3026  0.100  0.100 +0.000   0.000000
  1  2.3013  2.3017  0.105  0.105 +0.000   2.406674
  2  2.3015  2.3020  0.101  0.101 -0.000   2.408133
  3  2.3023  2.3024  0.100  0.101 -0.000   2.408786
  4  2.3023  2.3025  0.100  0.100 -0.000   2.409872
  5  2.3024  2.3026  0.100  0.100 -0.000   2.410422
  6  2.3024  2.3026  0.100  0.100 -0.000   2.410910
  7  2.3025  2.3026  0.100  0.100 -0.000   2.411933
  8  2.3025  2.3026  0.100  0.100 -0.000   2.411943
  9  2.3025  2.3026  0.100  0.100 -0.000   2.411963
 10  2.3025  2.3026  0.100  0.100 -0.000   2.411967
 11  2.3025  2.3026  0.100  0.100 -0.000   2.411967
 12  2.3025  2.3026  0.100  0.100 -0.000   2.412111
 13  2.3025  2.3026  0.100  0.100 -0.000   2.412118
 14  2.3025  2.3026  0.100  0.100 -0.000   2.412133
 15  2.3025  2.3026  0.100  0.100 -0.000   2.412133
 16  2.3025  2.3026  0.100  0.100 -0.000   2.412133
 17  2.3025  2.3026  0.100  0.100 -0.000   2.412134
 18  2.3025  2.3026  0.100  0.100 -0.000   2.412134
 19  2.3025  2.3026  0.100  0.100 -0.000   2.412134
 20  2.3025  2.3026  0.100  0.100 -0.000   2.412134
```

In [18]:

```python
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for h, lb in [(h_mup, 'μP'), (h_sp, 'SP')]:
    axes[0].plot(h['ep'], h['tr_l'], '-',  label=f'{lb} train')
    axes[0].plot(h['ep'], h['te_l'], '--', label=f'{lb} test')
    axes[1].plot(h['ep'], h['tr_a'], '-',  label=f'{lb} train')
    axes[1].plot(h['ep'], h['te_a'], '--', label=f'{lb} test')
    gap = [a - b for a, b in zip(h['tr_a'], h['te_a'])]
    axes[2].plot(h['ep'], gap, '-o', label=lb, ms=3)

axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('Loss')
axes[0].set_title('Loss'); axes[0].legend(fontsize=8); axes[0].grid(True, alpha=0.3)

axes[1].set_xlabel('Epoch'); axes[1].set_ylabel('Accuracy')
axes[1].set_title('Accuracy'); axes[1].legend(fontsize=8); axes[1].grid(True, alpha=0.3)

axes[2].set_xlabel('Epoch'); axes[2].set_ylabel('Train Acc - Test Acc')
axes[2].set_title('Generalization Gap'); axes[2].legend(); axes[2].grid(True, alpha=0.3)
axes[2].axhline(0, color='gray', ls=':', lw=0.8)

fig.suptitle(f'Training Curves (width={W})', fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```



In [ ]:

In [ ]:

In [ ]: